

Software Requirements Specification

for

application for keeping track of watering plants

Version 2.0

Prepared by Wiktor Mandra

Lodz University of Technology

09.05.2026

Table of Contents

Table of Contents

1. Introduction.....	4
1.1. Purpose	4
1.2. Document Conventions	4
1.2.1. Terminology:.....	4
1.2.2. Formatting:.....	4
1.3. Intended Audience and Reading Suggestions	4
1.4. Product Scope	4
1.5. References	5
2. Overall Description.....	5
2.1. Product Perspective.....	5
2.2. Product Features	5
2.2.1 Main Features.....	5
2.2.2. Use Case Diagram.....	6
2.3. User Classes and Characteristics	7
2.4. Operating Environment.....	7
2.5. Design and Implementation Constraints.....	7
2.6. User Documentation.....	7
2.7. Assumptions and Dependencies.....	7
3. System Features.....	7
3.1. «Feature Name».....	8
4. External Interface Requirements.....	8
4.1. User Interfaces Overview.....	8

4.2. Hardware Interfaces.....	8
4.3. Software Interfaces.....	8
5. System Features/Modules.....	8
5.1. «System Feature Name».....	9
5.1.2 Stimulus/Response Sequences.....	9
5.1.3 Functional Requirements.....	9
6. Nonfunctional Requirements.....	9
6.1. Performance	9
6.2. Security	9
6.3 Usability.....	9

Revision History

Name	Date	Reason for Changes	Version
Wiktor Mandra	01.05.2026	Functional and non-functional requirements definition	1.0
Wiktor Mandra	09.05.2026	Use case diagram	2.0

1. Introduction

1.1. Purpose

This document specifies software requirements for “GreeLendar”, a mobile application designed to help users track the watering, fertilization and general care of their plants. This app aims to simplify plant maintenance by providing reminders, logs and care recommendations based on plant species, environment, and user input.

1.2. Document Conventions

1.2.1. Terminology:

Plant Profile: A digital record of a user’s plant including species, age and care history.

Care Log: A timeline of actions (watering, fertilization, etc.) performed on a plant.

1.2.2. Formatting:

Requirements are labeled as REQ-xx (functional) or NF-xx (non-functional).

Priorities: High (H), Medium (M), Low (L).

1.3. Intended Audience and Reading Suggestions

Developers: Focus on technical constraints and interfaces, sections

Project Managers: Review scope, dependencies, and timelines, sections

Users: Understand features and user interactions, sections

Testers: Verify functional and non-functional requirements, sections

1.4. Product Scope

GreeLendar is a cross-platform app (Android/iOS/Web) that:

Tracks watering, fertilization and repotting schedules.

Provides care reminders based on plants species and environmental conditions (e.g., humidity, sunlight, temperature).

Logs historical data (e.g., growth photos, soil moisture).

Offers a database of plant care guides (e.g., water frequency, ideal light).

Supports multi-user households (shared plant care).

1.5. References

(1998). *IEEE Recommended Practice for Software Requirements Specifications* (IEEE Std 830-1998) [Review of *IEEE Recommended Practice for Software Requirements Specifications*]. IEEE. <https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=720574>

Trefle | *The plants API*. (2024). Trefle.io. <https://trefle.io/>

Google. (2023). *Material Design*. Material Design. <https://m3.material.io/>

2. Overall Description

2.1. Product Perspective

The GreeLendar is a standalone app which utilizes third party Trefle API and PostgreSQL for data backup and between user synchronization.

2.2. Product Features

2.2.1 Main Features

Plant profile: Add or edit plants with species name, photo, care guidelines, log information

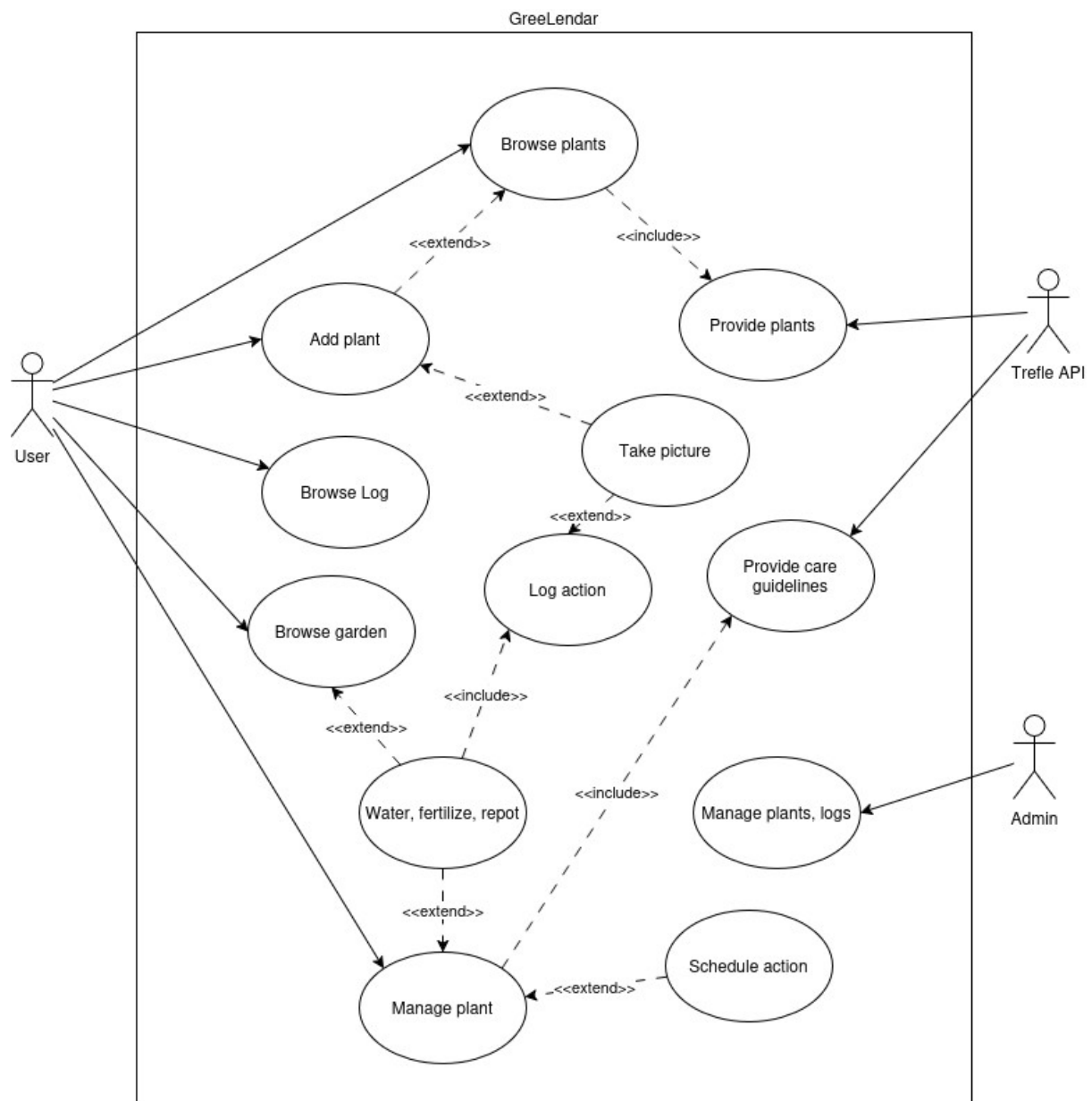
Care scheduling: Set cyclical reminders for action such as watering, repoting, etc.

Log actions: Save historical action taken to specific plant

Plants Database: Browse list of all plats list provided by Trefle

Take picture: Take picture when adding new plant or to confirm action taken

2.2.2. Use Case Diagram



The GreeLendar plant care system is designed to help and guide users in plantcare routines. It enables them to add plants to virtual garden schedule watering etc., check guidelines. The Trefle API provided all data regarding plants and how to care for them. Admin can manage all the actions.

2.3. User Classes and Characteristics

«Identify the various user classes (intended target for the product) that you anticipate will use this product. User classes may be differentiated based on frequency of use, subset of product functions used, technical expertise, security or privilege levels, educational level, or experience. Describe the pertinent characteristics of each user class. Certain requirements may pertain only to certain user classes. Distinguish the most important user classes for this product from those who are less important to satisfy.»

2.4. Operating Environment

«Describe the environment in which the software will operate, including the hardware platform, operating system and versions, and any other software components or applications with which it must peacefully coexist.»

2.5. Design and Implementation Constraints

«Describe any items or issues that will limit the options available to the developers. These might include: corporate or regulatory policies; hardware limitations (timing requirements, memory requirements); interfaces to other applications; specific technologies, tools, and databases to be used; parallel operations; language requirements; communications protocols; security considerations; design conventions or programming standards (for example, if the customer's organization will be responsible for maintaining the delivered software).»

2.6. User Documentation

«List the user documentation components (such as user manuals, on-line help, and tutorials) that will be delivered along with the software. Identify any known user documentation delivery formats or standards.»

2.7. Assumptions and Dependencies

«List any assumed factors (as opposed to known facts) that could affect the requirements stated in the specification. These could include third-party or commercial components that you plan to use, issues around the development or operating environment, or constraints. The project could be affected if these assumptions are incorrect, are not shared, or change. Also identify any dependencies the project has on external factors, such as software components that you intend to reuse from another project, unless they are already documented elsewhere (for example, in the vision and scope document or the project plan).»

3. System Features

«The purpose here is to give a brief introduction to the system features, so that §4 has something to reference. Then, the full explanation of the system features is given in §5, likely referencing §4. The numbering in §3 and §5 should match.»

3.1. «Feature Name»

«Brief description of the feature, generally 1-3 sentences»

3.«N». «Additional feature blocks as needed»

4. External Interface Requirements

4.1. User Interfaces Overview

«Describe the high level functionality of the system from the user's perspective. Explain how the user should be able to use your system to complete all the expected features and the feedback information that will be displayed for the user (for each screen the user will see). For example, specify whether it is GUI or text based interface and how at high level it would work. Discuss at high level how user will interact with the game such as clicking buttons, typing in some selection character, etc.»

«Describe the minimum requirements for the interface. This may include some sample screen images whether for text or GUI approach, any GUI standards or product family style guides that are to be followed, screen layout constraints, for GUI standard buttons and functions (e.g., help) that must appear on every screen, keyboard shortcuts, error message display standards, and so on»

4.2. Hardware Interfaces

«Describe the logical and physical characteristics of each interface between the software product and the hardware components of the system. This may include the supported device types, the nature of the data and control interactions between the software and the hardware, and communication protocols to be used.»

4.3. Software Interfaces

«Describe the connections between this product and other specific software components (name and version), including databases, interpreters, operating systems, tools, libraries, and integrated commercial components. Identify the data items or messages coming into the system and going out and describe the purpose of each. Describe the services needed and the nature of communications. Identify data that will be shared across software components. If the data sharing mechanism must be implemented in a specific way (for example, use of a global data area in a multitasking operating system), specify this as an implementation constraint.»

5. System Features/Modules

«This template illustrates organizing the functional requirements for the product by system features, the major services provided by the product.»

5.1. «System Feature Name»

5.1.1 Description and Priority

«Provide a short description of the feature. Give its priority, too.»

5.1.2 Stimulus/Response Sequences

«"Stimulus" or "Condition"»: «Describe the stimulus or condition»

Response: «Describe the response»

«Additional Stimulus/Response pairs as needed»

5.1.3 Functional Requirements

REQ-1.1: The app shall allow users to add plant by providing at least species name

REQ-1.2: The app shall support uploading (JPEG/PNG) images for plant profiles

REQ-2.1: The app shall allow users to set custom intervals for each care task

REQ-2.2: The app shall send notification 1 hour before the scheduled task

REQ-3.1: The app shall log date, time, and user for each care action

REQ-3.2: The app shall display timeline of past actions for each plant.

6. Nonfunctional Requirements

6.1. Performance

NF-1.1: The app shall load dashboard in <2 seconds

NF-1.2: The app shall support up to 100 plants per user without performance issues

6.2. Security

NF-2.1: The app shall require authentication for all sensitive actions (e.g., deleting plants)

6.3 Usability

NF-3.1 The app shall support dark mode